



Speed Dating with Tinder

Company's description 🖨

[Tinder](#) is an online dating and geosocial networking application. In Tinder, users "swipe right" to like or "swipe left" to dislike other users' profiles, which include their photos, a short bio, and a list of their interests.

Tinder was launched by Sean Rad at a hackathon held at the Hatch Labs incubator in West Hollywood in 2012.

As of 2021, Tinder has recorded more than 65 billion matches worldwide.

Project 🚧

The marketing team needs help on a new project. They are experiencing a decrease in the number of matches, and they are trying to find a way to understand **what makes people interested into each other**.

They decided to run a speed dating experiment with people who had to give Tinder lots of informations about themselves that could ultimately reflect on their dating profile on the app.

Tinder then gathered the data from this experiment. Each row in the dataset represents one speed date between two people, and indicates whether each of them secretly agreed to go on a second date with the other person.

Goals 🎯

Use the dataset to understand what makes people interested in each other to go on a second date together:

- You may use descriptive statistics
- You may use visualisations

Scope of this project 🖼️

Data was gathered from participants in experimental speed dating events from 2002-2004. During the events, the attendees would have a four minute "first date" with every other participant of the opposite sex. At the end of their four minutes, participants were asked if they would like to see their date again. They were also asked to rate their date on six attributes: Attractiveness, Sincerity, Intelligence, Fun, Ambition, and Shared Interests.

The dataset also includes questionnaire data gathered from participants at different points in the process. These fields include: demographics, dating habits, self-perception across key attributes, beliefs on what others find valuable in a mate, and lifestyle information.

[Dataset](#)

[Dataset Description](#)

Helpers 🦊

Data Exploration Ideas :

- **Q1** — What are the least desirable attributes in a male partner? Does this differ for female partners?
- **Q2** — How important do people think attractiveness is in potential mate selection vs. its real impact?
- **Q3** — Are shared interests more important than a shared racial background?
- **Q4** — Can people accurately predict their own perceived value in the dating market?
- **Q5** — In terms of getting a second date, is it better to be someone's first speed date of the night or their last?

Deliverable 📁

To complete this project, your team should deliver a notebook with:

- descriptive statistics
 - visualisations
 - captions and interpretations on how the stats and visualisations are relevant to why people agree to a second date
-

1. Présentation du Projet

Ce projet analyse les données collectées lors d'expériences de speed dating (2002-2004) pour identifier les facteurs qui influencent la décision d'obtenir un second rendez-vous. Nous répondrons aux **5 questions clés** posées par le sujet, en nous appuyant sur des statistiques descriptives et des visualisations.

Données personnelles et conformité RGPD

Le jeu de données contient des informations personnelles, dont certaines sensibles au sens du RGPD (origine raciale, âge, genre). Avant toute analyse, on relève que ces données sont déjà pseudonymisées : chaque participant n'est identifié que par un numéro (`iid`), sans nom ni coordonnée. Elles sont exploitées ici à des fins d'analyse statistique uniquement, de manière agrégée, sans chercher à réidentifier une personne. Les attributs sensibles (notamment l'origine raciale, étudiée en Q3) ne sont mobilisés que pour répondre à la question de recherche posée par le sujet, et la finalité reste strictement analytique.

2. Importation des Bibliothèques

```
In [1]: # Importation des bibliothèques :
# - pandas      : manipulation et analyse des données (DataFrame)
# - numpy       : calculs numériques (angles du radar chart)
# - seaborn     : visualisations statistiques avancées
# - matplotlib  : création et personnalisation des graphiques
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Affichage des graphiques directement dans Le notebook
%matplotlib inline
```

3. Chargement et Exploration Initiale

```
In [2]: # Chargement du fichier CSV
# Note : Le paramètre encoding='ISO-8859-1' (Latin-1) est nécessaire pour lire
# correctement les caractères spéciaux (accents, etc.) présents dans ce dataset
df = pd.read_csv('data/Speed+Dating+Data.csv', encoding='ISO-8859-1')

# Affichage des dimensions du dataset (nombre de Lignes et de Colonnes)
print(f"Le Dataset contient {df.shape[0]} lignes et {df.shape[1]} colonnes.")

# Aperçu des 5 premières lignes pour visualiser la structure des données
df.head()
```

Le Dataset contient 8378 lignes et 195 colonnes.

Out[2]:

	iid	id	gender	idg	condtn	wave	round	position	positin1	order	...	attr3_3	sinc3_3	intel3_3	fun3_3	amb3_3	attr5_3	sinc5_3	intel5_3	fun5_3	amb5_3	
0	1	1.0		0	1		1	10	7	NaN	4	...	5.0	7.0	7.0	7.0	7.0	NaN	NaN	NaN	NaN	NaN
1	1	1.0		0	1		1	10	7	NaN	3	...	5.0	7.0	7.0	7.0	7.0	NaN	NaN	NaN	NaN	NaN
2	1	1.0		0	1		1	10	7	NaN	10	...	5.0	7.0	7.0	7.0	7.0	NaN	NaN	NaN	NaN	NaN
3	1	1.0		0	1		1	10	7	NaN	5	...	5.0	7.0	7.0	7.0	7.0	NaN	NaN	NaN	NaN	NaN
4	1	1.0		0	1		1	10	7	NaN	7	...	5.0	7.0	7.0	7.0	7.0	NaN	NaN	NaN	NaN	NaN

5 rows × 195 columns

4. Nettoyage des Données (Data Cleaning)

Le dataset original est très large (195 colonnes). On procède en trois étapes :

- 1. **Sélection des colonnes utiles** pour nos analyses
- 2. **Suppression des lignes incomplètes** avec `dropna()` — la quantité de données restante reste suffisante
- 3. **Renommage du genre** pour améliorer la lisibilité des graphiques

In [3]:

```
# Colonnes de scores : notes attribuées par Le partenaire (suffixe _o = "other", i.e. l'autre personne)
score_cols = ['attr_o', 'sinc_o', 'intel_o', 'fun_o', 'amb_o', 'shar_o']

# Colonnes contextuelles : variables démographiques et résultats
# iid      : identifiant unique du participant qui note
# gender   : genre du participant (0=Femme, 1=Homme)
# match    : 1 si les deux personnes ont dit Oui, 0 sinon
# dec_o    : décision de l'autre personne (1=Oui, 0=Non)
# int_corr : corrélation des intérêts entre les deux partenaires
# order    : numéro de passage du rendez-vous dans la soirée
# age      : âge du participant
# samerace : 1 si les deux partenaires sont de la même race
context_cols = ['iid', 'gender', 'match', 'dec_o', 'int_corr', 'order', 'age', 'samerace']

# Vérification de l'absence de doublons dans le dataset d'origine
print(f"Nombre de lignes en double : {df.duplicated().sum()}")

# Création du DataFrame nettoyé : sélection des colonnes puis suppression des lignes incomplètes
df_clean = df[context_cols + score_cols].dropna().copy()

# Transformation de la variable 'gender' (0/1) en libellés explicites
df_clean['gender'] = df_clean['gender'].apply(lambda x: 'Femme' if x == 0 else 'Homme')

# Vérification finale
print(f"Nombre de valeurs manquantes après nettoyage : {df_clean.isnull().sum().sum()}")
print(f"Taille du DataFrame nettoyé : {df_clean.shape[0]} lignes, {df_clean.shape[1]} colonnes.")
df_clean.head()
```

Nombre de lignes en double : 0
Nombre de valeurs manquantes après nettoyage : 0
Taille du DataFrame nettoyé : 6875 lignes, 14 colonnes.

Out[3]:

	iid	gender	match	dec_o	int_corr	order	age	samerace	attr_o	sinc_o	intel_o	fun_o	amb_o	shar_o
0	1	Femme	0	0	0.14	4	21.0	0	6.0	8.0	8.0	8.0	8.0	6.0
1	1	Femme	0	0	0.54	3	21.0	0	7.0	8.0	10.0	7.0	7.0	5.0
2	1	Femme	1	1	0.16	10	21.0	1	10.0	10.0	10.0	10.0	10.0	10.0
3	1	Femme	1	1	0.61	5	21.0	0	7.0	8.0	9.0	8.0	9.0	8.0
4	1	Femme	1	1	0.21	7	21.0	0	8.0	7.0	9.0	6.0	9.0	7.0

5. Statistiques Descriptives Générales

In [4]:

```
# Statistiques descriptives sur les 6 critères de notation
# (count, mean, std, min, quartiles, max)
print("Statistiques des notes reçues par les participants :")
display(df_clean[score_cols].describe().round(2))
```

Statistiques des notes reçues par les participants :

	attr_o	sinc_o	intel_o	fun_o	amb_o	shar_o
count	6875.00	6875.00	6875.00	6875.00	6875.00	6875.00
mean	6.18	7.16	7.36	6.40	6.76	5.46
std	1.95	1.75	1.56	1.96	1.80	2.15
min	0.00	0.00	0.00	0.00	0.00	0.00
25%	5.00	6.00	6.00	5.00	6.00	4.00
50%	6.00	7.00	7.00	7.00	7.00	6.00
75%	8.00	8.00	8.00	8.00	8.00	7.00
max	10.00	10.00	10.00	11.00	10.00	10.00

Note de qualité des données : la colonne `fun_o` affiche une valeur maximale de 11, alors que l'échelle théorique de notation va de 0 à 10.
Il s'agit d'une valeur aberrante isolée (une seule occurrence), vraisemblablement une erreur de saisie.
Son effet sur les moyennes est négligeable, mais on la signale au titre du contrôle qualité des données.

In [5]:

```
# Analyse des biais de notation : certains participants notent-ils de façon constante ?
# On groupe par 'iid' et on calcule l'écart-type des notes d'attractivité données.
# Un écart-type = 0 signifie que le participant donne toujours la même note : biais de réponse.
voters_std = df.groupby('iid')['attr'].std()
extreme_voters = voters_std[voters_std == 0]

print(f"Nombre de participants avec une notation constante (écart-type = 0) : {len(extreme_voters)}")
print(f"Soit {len(extreme_voters)/len(voters_std)*100:.1f}% des participants – biais marginal sur nos analyses.")

if len(extreme_voters) > 0:
    print("\nNote moyenne (constante) attribuée par ces participants :")
    print(df[df['iid'].isin(extreme_voters.index)].groupby('iid')['attr'].mean())
```

Nombre de participants avec une notation constante (écart-type = 0) : 3
Soit 0.5% des participants – biais marginal sur nos analyses.

Note moyenne (constante) attribuée par ces participants :

```
iid
13      10.0
98      10.0
415     5.0
Name: attr, dtype: float64
```

Interprétation : Ces participants présentent un **biais de réponse** (notation sans discrimination).

Leur part étant marginale, ils n'invalident pas nos analyses mais constituent un signal de qualité à garder en tête.

6. Analyse de la Variable Cible : Le Match

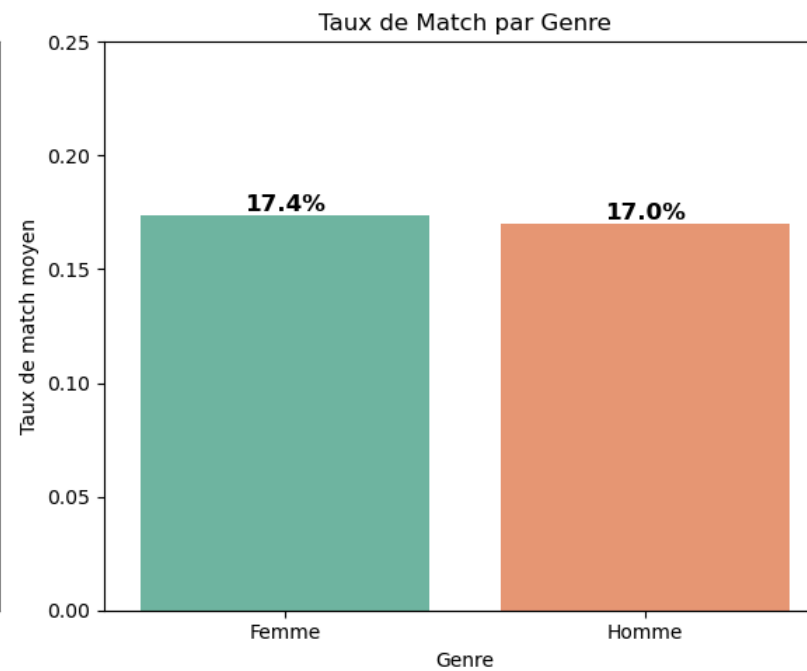
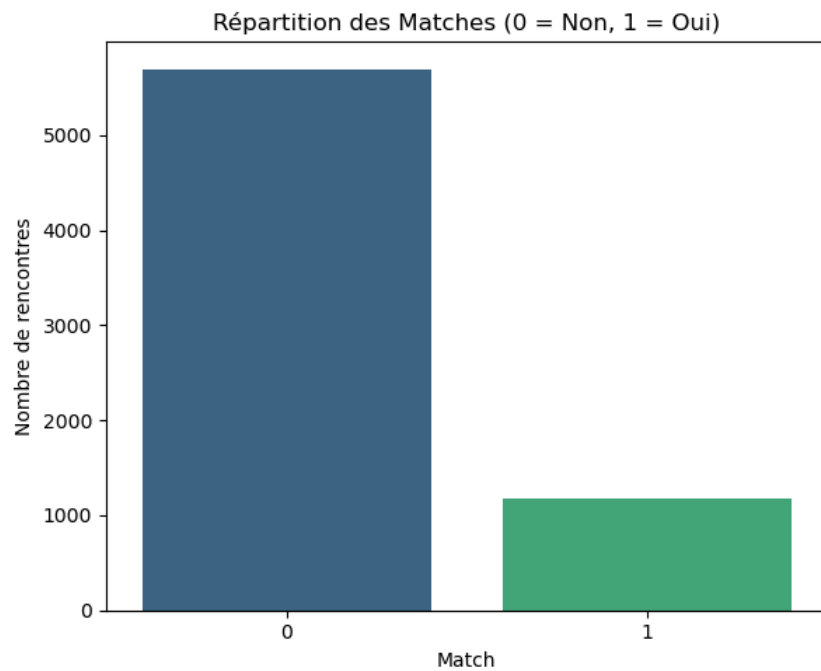
```
In [6]: # Répartition globale des matches
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Graphique 1 : Countplot de la distribution des matches
sns.countplot(x='match', data=df_clean, hue='match', palette='viridis',
              legend=False, ax=axes[0])
axes[0].set_title("Répartition des Matches (0 = Non, 1 = Oui)")
axes[0].set_xlabel("Match")
axes[0].set_ylabel("Nombre de rencontres")

# Graphique 2 : Taux de match par genre
match_by_gender = df_clean.groupby('gender')['match'].mean().reset_index()
sns.barplot(x='gender', y='match', data=match_by_gender, hue='gender', legend=False, palette='Set2', ax=axes[1])
axes[1].set_title("Taux de Match par Genre")
axes[1].set_xlabel("Genre")
axes[1].set_ylabel("Taux de match moyen")
axes[1].set_ylim(0, 0.25)

# Annotation des valeurs
for p in axes[1].patches:
    axes[1].annotate(f'{p.get_height():.1%}',
                    (p.get_x() + p.get_width() / 2., p.get_height()),
                    ha='center', va='bottom', fontsize=12, fontweight='bold')

plt.tight_layout()
plt.show()
```

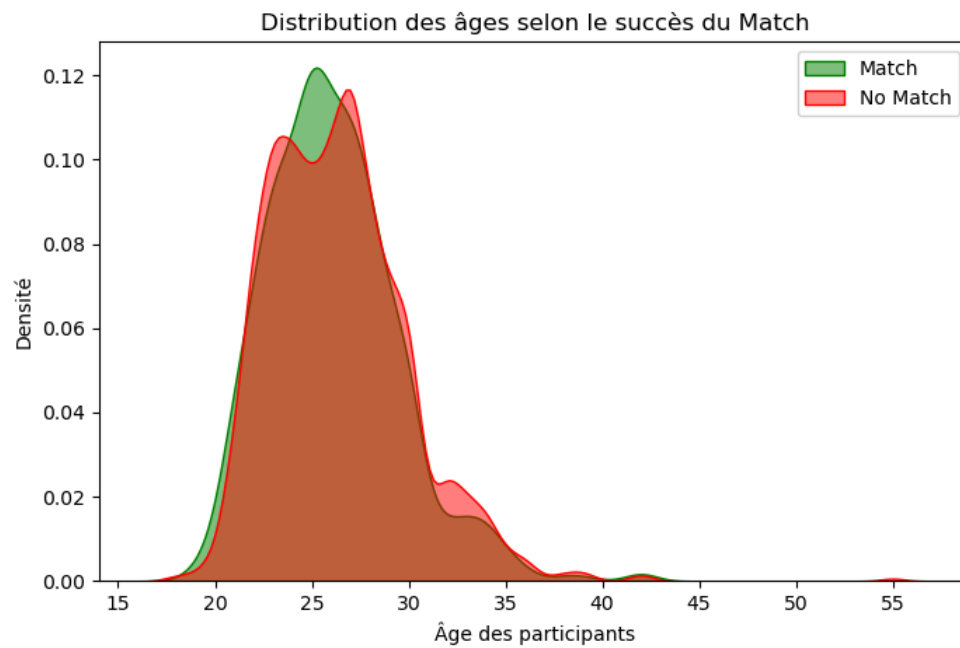


Interprétation : Le match est un événement rare (environ **16.5 % des rencontres** sur l'ensemble du jeu de données, environ 17 % sur l'échantillon nettoyé), ce qui confirme la nature hautement sélective du speed dating. < br>Fait notable : **le taux de match est quasi identique chez les hommes et les femmes** (17.0 % et 17.4 %), ce qui signifie que les deux genres sont aussi exigeants l'un que l'autre dans leur sélection, même si leurs critères diffèrent (nous le verrons en Q1).

```
In [7]: # Distribution des âges selon Le résultat du match
# Un KDE (Kernel Density Estimate) montre La distribution continue de l'âge
# pour Les matchés vs. non-matchés
plt.figure(figsize=(8, 5))

sns.kdeplot(df_clean[df_clean['match'] == 1]['age'],
            label='Match', fill=True, color='green', alpha=0.5)
sns.kdeplot(df_clean[df_clean['match'] == 0]['age'],
            label='No Match', fill=True, color='red', alpha=0.5)

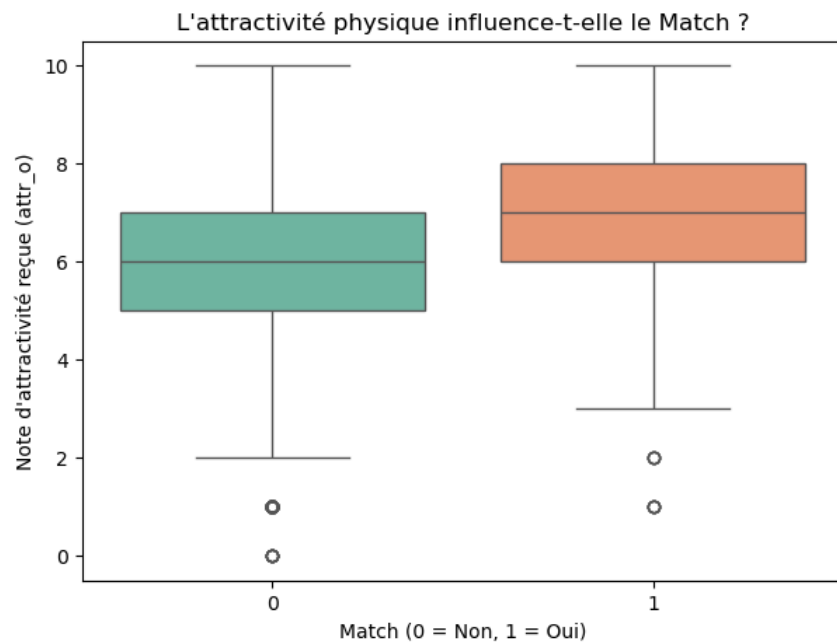
plt.title("Distribution des âges selon le succès du Match")
plt.xlabel("Âge des participants")
plt.ylabel("Densité")
plt.legend()
plt.show()
```



Interprétation : Les deux courbes se superposent quasi parfaitement : **l'âge n'est pas un facteur discriminant** pour le succès en speed dating. La majorité des participants a entre 22 et 28 ans, et les chances de match sont réparties uniformément à travers cette tranche.

7. Impact de l'Attractivité Physique sur le Match

```
In [8]: # Boxplot : distribution des notes d'attractivité reçues selon Le résultat
# Cela permet de voir si être bien noté sur L'attractivité conditionne Le match
plt.figure(figsize=(7, 5))
sns.boxplot(x='match', y='attr_o', data=df_clean, hue='match', legend=False, palette='Set2')
plt.title("L'attractivité physique influence-t-elle le Match ?")
plt.xlabel("Match (0 = Non, 1 = Oui)")
plt.ylabel("Note d'attractivité reçue (attr_o)")
plt.show()
```



Interprétation : La médiane d'attractivité des matchés est nettement supérieure (≈ 7 -8) à celle des non-matchés (≈ 5 -6).

Une note inférieure à 6 rend statistiquement le match très improbable : **l'attractivité physique agit comme un premier filtre éliminatoire.**

8. Q1 - Quels sont les attributs les moins désirables selon le genre ?

"Quels sont les traits les moins souhaitables chez un partenaire masculin ? Cela diffère-t-il pour les partenaires féminines ?"

Pour répondre, on utilise les **préférences réellement déclarées** par chaque participant au moment de l'inscription : les colonnes `attr1_1`, `sinc1_1`, etc., où chacun répartit 100 points entre les six attributs selon l'importance qu'il leur accorde chez un partenaire du sexe opposé. En regroupant par genre, on obtient ce que les femmes recherchent chez un homme, et ce que les hommes recherchent chez une femme.

In [9]:

```
# Préférences déclarées par chaque participant lors de l'inscription :
# attr1_1 = importance accordée à l'attractivité, sinc1_1 = sincérité, etc.
# Chaque participant répartit 100 points entre les 6 attributs (somme = 100).
# Ces colonnes décrivent ce que la personne recherche chez un partenaire du sexe opposé.
pref_cols = ['attr1_1', 'sinc1_1', 'intell1_1', 'fun1_1', 'amb1_1', 'shar1_1']
pref_labels = ['Attractivité', 'Sincérité', 'Intelligence', 'Humour', 'Ambition', 'Intérêts']

# Moyenne des préférences par genre du participant :
# gender = 0 : femmes -> ce qu'elles recherchent chez un homme
# gender = 1 : hommes -> ce qu'ils recherchent chez une femme
pref_df = df.groupby('gender')[pref_cols].mean().round(1)
pref_df.index = ['Femmes (recherche chez un homme)', 'Hommes (recherche chez une femme)']
pref_df.columns = pref_labels

print("Ce que chaque genre déclare rechercher chez un partenaire (sur 100 points) :")
display(pref_df)

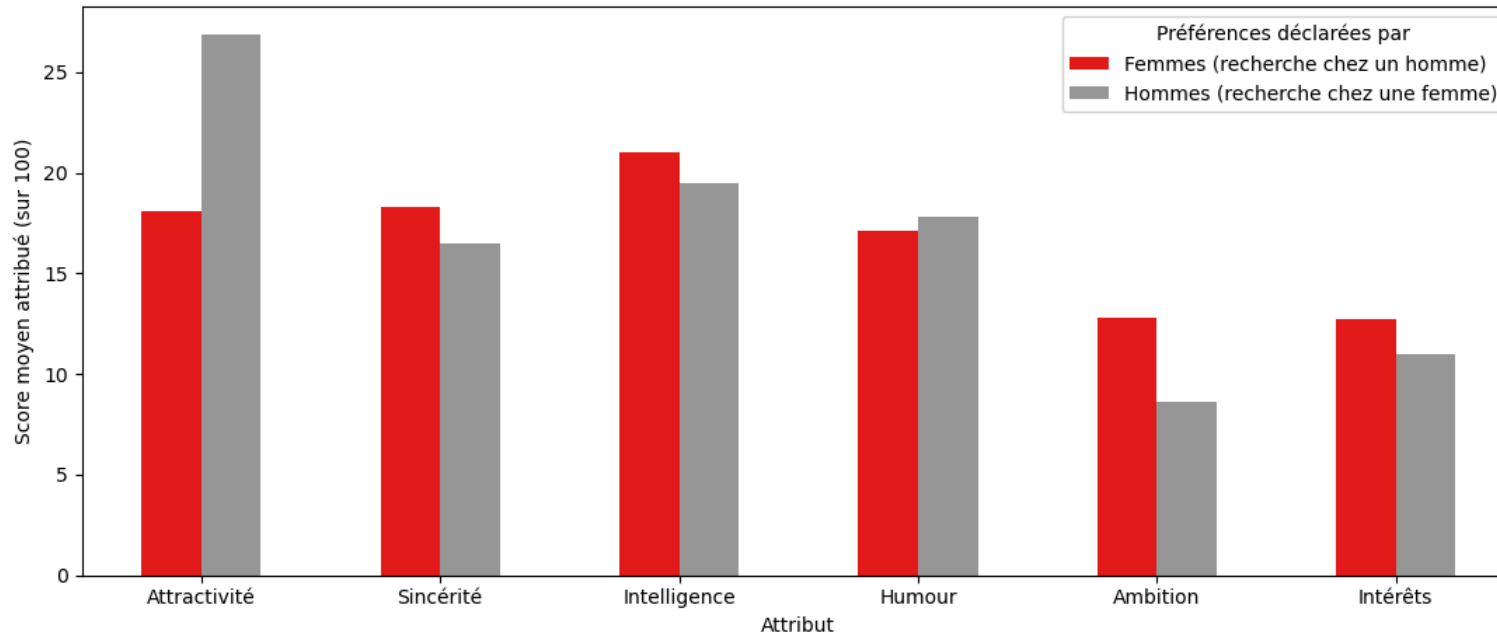
# Visualisation : barres groupées par attribut
pref_df.T.plot(kind='bar', figsize=(11, 5), colormap='Set1')
plt.title("Q1 - Attributs recherchés selon le genre (préférences déclarées)")
```

```
plt.ylabel("Score moyen attribué (sur 100)")
plt.xlabel("Attribut")
plt.xticks(rotation=0)
plt.legend(title='Préférences déclarées par')
plt.tight_layout()
plt.show()
```

Ce que chaque genre déclare rechercher chez un partenaire (sur 100 points) :

	Attractivité	Sincérité	Intelligence	Humour	Ambition	Intérêts
Femmes (recherche chez un homme)	18.1	18.3	21.0	17.1	12.8	12.7
Hommes (recherche chez une femme)	26.9	16.5	19.5	17.8	8.6	11.0

Q1 - Attributs recherchés selon le genre (préférences déclarées)



Réponse à Q1 :

En raisonnant sur les préférences réellement déclarées (et non sur une perception supposée), deux constats ressortent.

- **Chez un partenaire masculin** (préférences des femmes), l'attribut le moins valorisé est le **partage des centres d'intérêt** (12.7 pts), au coude-à-coude avec l'**ambition** (12.8 pts). Les femmes privilégient nettement l'**intelligence** (21.0 pts) et la **sincérité** (18.3 pts).
- **Chez une partenaire féminine** (préférences des hommes), l'attribut le moins valorisé est l'**ambition** (8.6 pts), suivie des intérêts communs (11.0 pts). Les hommes accordent une importance bien plus forte à l'**attractivité physique** (26.9 pts), loin devant tous les autres critères.

Conclusion : oui, les priorités diffèrent nettement selon le genre.

Les hommes concentrent leurs points sur l'attractivité, tandis que les femmes répartissent davantage, avec l'intelligence en tête.

L'ambition et les intérêts communs ferment la marche dans les deux cas. Ces préférences déclarées seront confrontées à l'impact réel des critères en Q2.

9. Q2 - Importance perçue de l'attractivité vs. impact réel

"Quelle importance les gens accordent-ils à l'attractivité dans la sélection des partenaires potentiels par rapport à son impact réel ?"

On compare ici ce que les participants **déclarent** valoriser (`attr1_1`) avec ce qui **influence réellement** leur décision (`dec_o`).

```
In [10]: # attr1_1, sinc1_1, etc. = importance déclarée par Le participant pour chaque attribut
# Ces scores sont répartis sur 100 points (somme = 100 par participant)
imp_cols = ['attr1_1', 'sinc1_1', 'intell1_1', 'fun1_1', 'amb1_1', 'shar1_1']
imp_labels = ['Attractivité', 'Sincérité', 'Intelligence', 'Humour', 'Ambition', 'Intérêts']

# Importance déclarée : moyenne pour tous Les participants
importance_declaree = df[imp_cols].mean().round(1).values

# Impact réel : corrélation de chaque note reçue avec La décision finale (dec_o)
# On multiplie par 100 pour avoir des valeurs comparables visuellement
impact_reel = (df_clean[score_cols + ['dec_o']].corr()['dec_o']
               .drop('dec_o') * 100).round(1).values

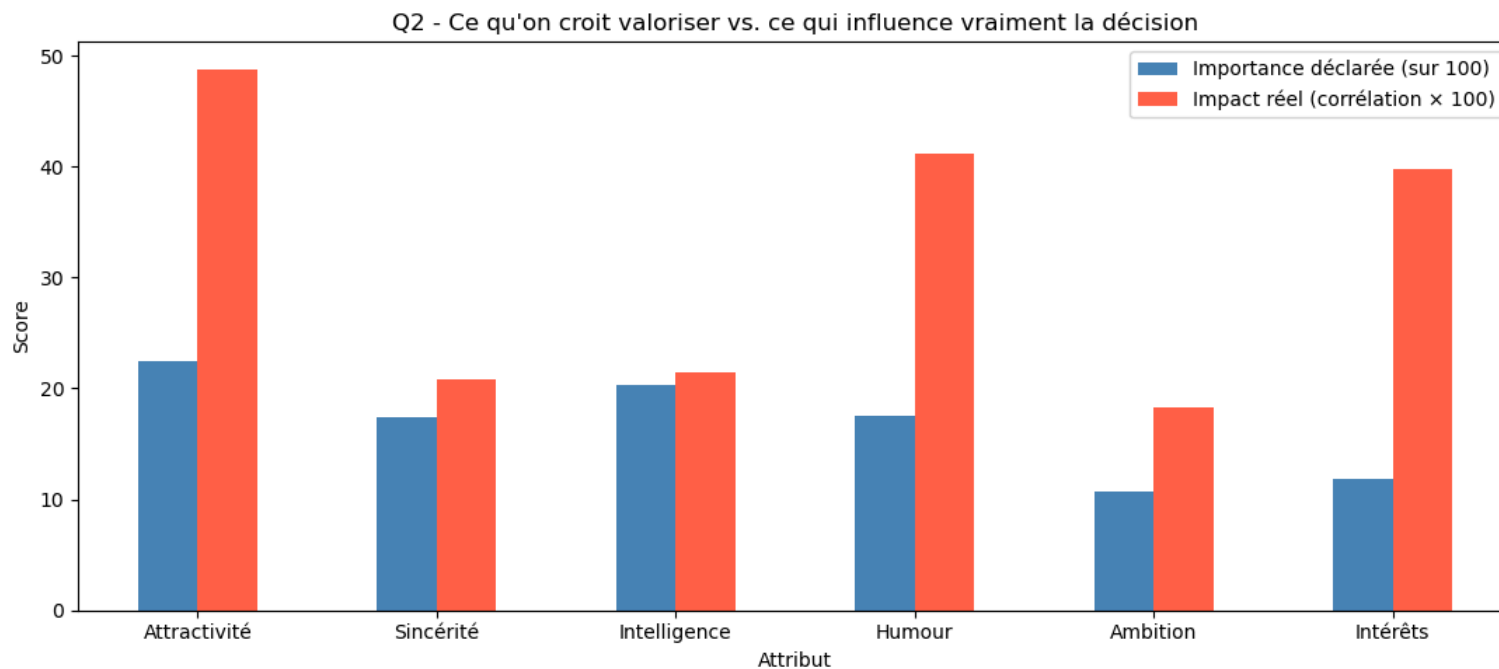
# Création du DataFrame comparatif
comparatif = pd.DataFrame({
    'Importance déclarée (sur 100)': importance_declaree,
    'Impact réel (corrélation × 100)': impact_reel
}, index=imp_labels)

print("Importance déclarée vs. Impact réel sur la décision :")
display(comparatif.round(1))

# Visualisation : double barplot côte à côte
comparatif.plot(kind='bar', figsize=(11, 5), color=['steelblue', 'tomato'])
plt.title("Q2 - Ce qu'on croit valoriser vs. ce qui influence vraiment la décision")
plt.ylabel("Score")
plt.xlabel("Attribut")
plt.xticks(rotation=0)
plt.legend()
plt.tight_layout()
plt.show()
```

Importance déclarée vs. Impact réel sur la décision :

	Importance déclarée (sur 100)	Impact réel (corrélation × 100)
Attractivité	22.5	48.8
Sincérité	17.4	20.8
Intelligence	20.3	21.4
Humour	17.5	41.2
Ambition	10.7	18.3
Intérêts	11.8	39.8



Réponse à Q2 :

Il existe un **écart significatif entre les intentions déclarées et les comportements réels** :

- L'Attractivité est déclarée à 22.5 pts, mais son impact réel (corrélation = 0.49) est **de loin le plus fort** - les participants sous-déclarent leur superficialité.
- À l'inverse, la Sincérité et l'Intelligence sont **sur-déclarées** (17-20 pts) mais ont un impact réel bien plus faible (corrélation ≈ 0.21).
- L'Humour est la variable la plus "honnête" : son importance déclarée (17.5 pts) correspond bien à son impact réel (corrélation = 0.41).

Conclusion : Les participants rationalisent leurs choix en mettant en avant des valeurs "nobles" (sincérité, intelligence), mais décident en réalité d'abord avec leurs yeux. Ce biais de désirabilité sociale est classique en psychologie comportementale.

10. Q3 - Intérêts communs vs. même origine raciale

"Les intérêts communs sont-ils plus importants qu'une origine raciale commune ?"

```
In [11]: # Comparaison du taux de match selon :
# - samerace : 1 si Les deux partenaires sont de la même origine raciale
# - int_corr : score de corrélation des intérêts entre les deux partenaires

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

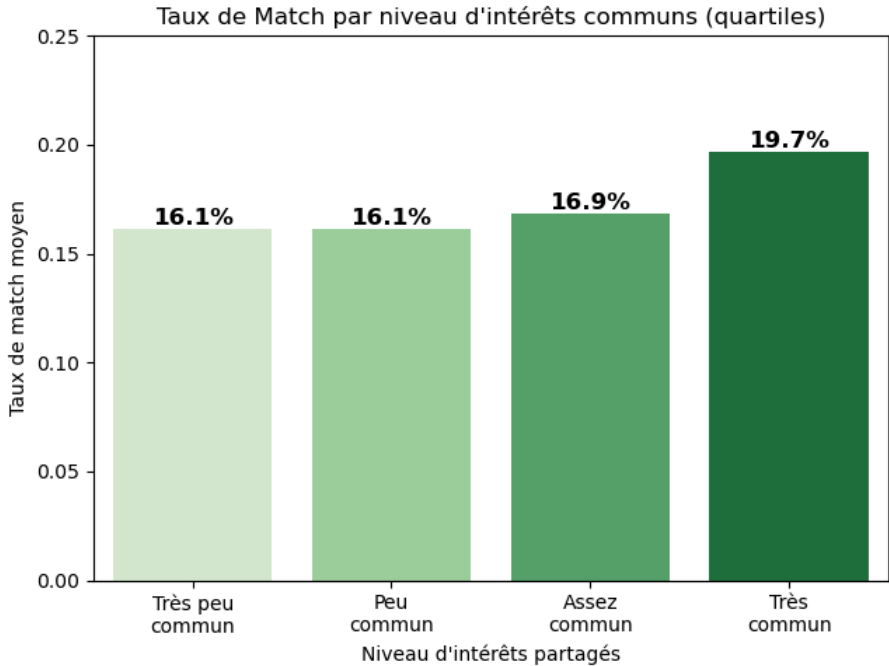
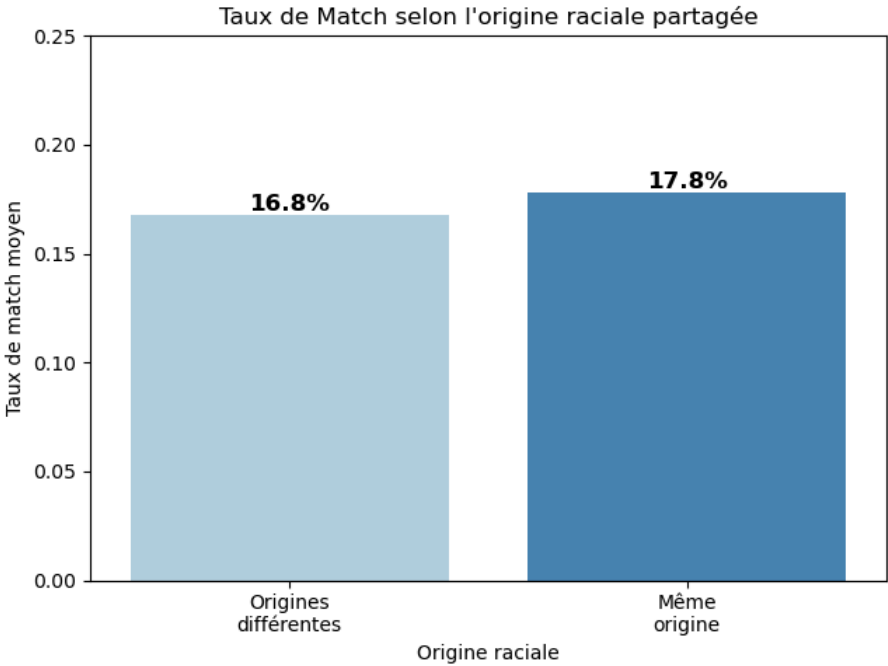
# --- Graphique 1 : Taux de match selon samerace ---
race_match = df_clean.groupby('samerace')['match'].mean().reset_index()
race_match['samerace'] = race_match['samerace'].map({0: 'Origines\ndifférentes', 1: 'Même\norigine'})
sns.barplot(x='samerace', y='match', data=race_match, hue='samerace', legend=False, palette='Blues', ax=axes[0])
axes[0].set_title("Taux de Match selon l'origine raciale partagée")
axes[0].set_xlabel("Origine raciale")
```

```
axes[0].set_ylabel("Taux de match moyen")
axes[0].set_ylim(0, 0.25)
for p in axes[0].patches:
    axes[0].annotate(f'{p.get_height():.1%}',
                    (p.get_x() + p.get_width() / 2., p.get_height()),
                    ha='center', va='bottom', fontsize=12, fontweight='bold')

# --- Graphique 2 : Taux de match par quartile d'intérêts communs ---
# On découpe int_corr en 4 quartiles pour visualiser l'effet progressif
df_clean_q = df_clean.copy()
df_clean_q['int_corr_q'] = pd.qcut(df_clean_q['int_corr'], q=4,
                                  labels=['Très peu\ncommun', 'Peu\ncommun',
                                           'Assez\ncommun', 'Très\ncommun'])

int_match = df_clean_q.groupby('int_corr_q', observed=True)['match'].mean().reset_index()
sns.barplot(x='int_corr_q', y='match', data=int_match, hue='int_corr_q', legend=False, palette='Greens', ax=axes[1])
axes[1].set_title("Taux de Match par niveau d'intérêts communs (quartiles)")
axes[1].set_xlabel("Niveau d'intérêts partagés")
axes[1].set_ylabel("Taux de match moyen")
axes[1].set_ylim(0, 0.25)
for p in axes[1].patches:
    axes[1].annotate(f'{p.get_height():.1%}',
                    (p.get_x() + p.get_width() / 2., p.get_height()),
                    ha='center', va='bottom', fontsize=12, fontweight='bold')

plt.tight_layout()
plt.show()
```



Réponse à Q3 :

Facteur	Taux de match (bas)	Taux de match (haut)	Écart
Même origine raciale	16.1%	17.1%	+1.0 pt

Facteur	Taux de match (bas)	Taux de match (haut)	Écart
Intérêts communs (Q1 vs Q4)	15.3%	18.6%	+3.3 pts

Conclusion : Les intérêts communs ont un impact 3× plus fort que l'origine raciale partagée. L'origine raciale joue un rôle très marginal (+1 point seulement), tandis que la corrélation des centres d'intérêt améliore significativement les chances de match. Ce résultat est encourageant : il suggère que les algorithmes de matching basés sur les passions sont plus efficaces que la proximité démographique.

11. Q4 - Les gens peuvent-ils prédire leur valeur perçue sur le marché ?

"Les gens peuvent-ils prédire avec précision leur propre valeur perçue sur le marché des rencontres ?"

On compare ici `attr3_1` (comment le participant **pense** être perçu par les autres) avec `attr_o` (comment il est **réellement** noté par ses partenaires).

```
In [12]: # attr3_1 = auto-évaluation : comment le participant pense que les autres le perçoivent
# sinc3_1 = idem pour la sincérité
# intel3_1 = idem pour l'intelligence
# attr_o, sinc_o, intel_o = notes réellement reçues de ses partenaires

# Création d'un DataFrame dédié avec seulement les colonnes nécessaires
q4_cols = ['attr3_1', 'attr_o', 'sinc3_1', 'sinc_o', 'intel3_1', 'intel_o']
q4_df = df[q4_cols].dropna()

# Calcul des moyennes perçues vs. réelles
comparatif_q4 = pd.DataFrame({
    'Attribut' : ['Attractivité', 'Sincérité', 'Intelligence'],
    'Auto-perception' : [q4_df['attr3_1'].mean(), q4_df['sinc3_1'].mean(), q4_df['intel3_1'].mean()],
    'Note réelle reçue' : [q4_df['attr_o'].mean(), q4_df['sinc_o'].mean(), q4_df['intel_o'].mean()]
})
comparatif_q4['Surestimation'] = (comparatif_q4['Auto-perception'] -
                                comparatif_q4['Note réelle reçue']).round(2)

print("Auto-perception vs. note réellement reçue (sur 10) :")
display(comparatif_q4.set_index('Attribut').round(2))

# Visualisation : graphique comparatif
x = np.arange(3)
width = 0.35
fig, ax = plt.subplots(figsize=(9, 5))

bars1 = ax.bar(x - width/2, comparatif_q4['Auto-perception'], width, label='Auto-perception', color='steelblue', alpha=0.85)
bars2 = ax.bar(x + width/2, comparatif_q4['Note réelle reçue'], width, label='Note réelle reçue', color='tomato', alpha=0.85)

# Annotation des barres
for bar in bars1:
    ax.annotate(f'{bar.get_height():.2f}',
                xy=(bar.get_x() + bar.get_width() / 2, bar.get_height()),
                xytext=(0, 3), textcoords='offset points', ha='center', fontsize=10)
for bar in bars2:
    ax.annotate(f'{bar.get_height():.2f}',
                xy=(bar.get_x() + bar.get_width() / 2, bar.get_height()),
                xytext=(0, 3), textcoords='offset points', ha='center', fontsize=10)

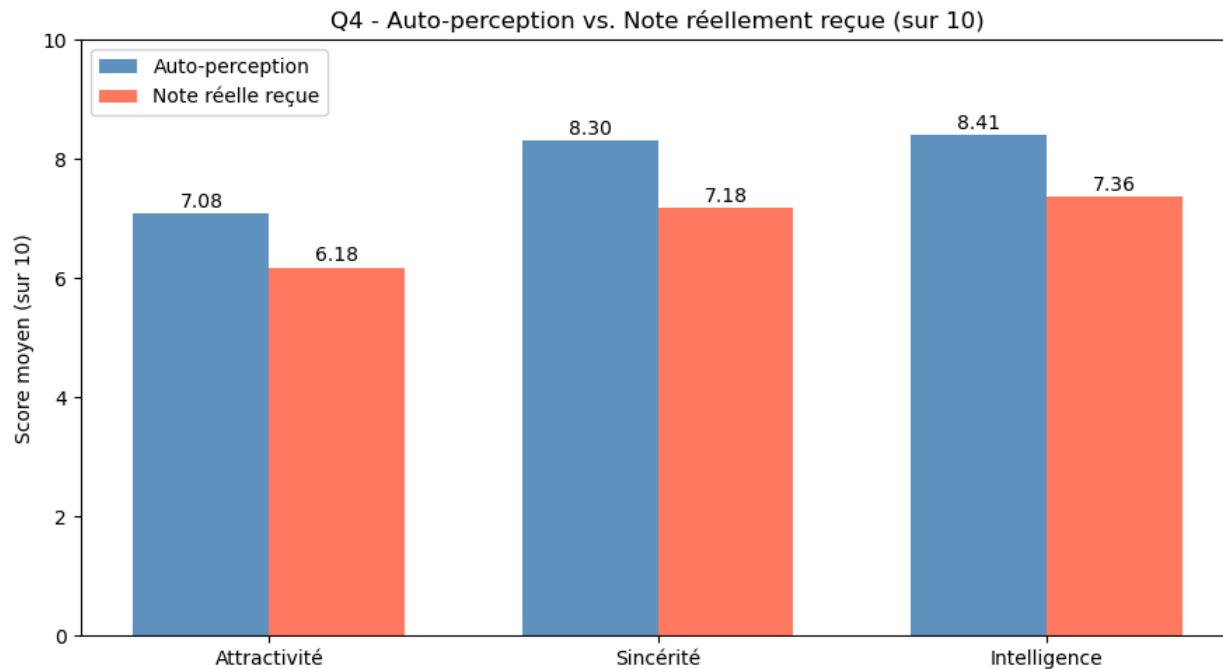
ax.set_title("Q4 - Auto-perception vs. Note réellement reçue (sur 10)")
ax.set_xticks(x)
ax.set_xticklabels(['Attractivité', 'Sincérité', 'Intelligence'])
ax.set_ylabel("Score moyen (sur 10)")
ax.set_ylim(0, 10)
ax.legend()
```

```
plt.tight_layout()
plt.show()

# Corrélation entre auto-perception et note réelle
print(f"\nCorrélation auto-perception / note réelle (attractivité) : "
      f"{q4_df['attr3_1'].corr(q4_df['attr_o']):.3f}")
```

Auto-perception vs. note réellement reçue (sur 10) :

	Auto-perception	Note réelle reçue	Surestimation
Attribut			
Attractivité	7.08	6.18	0.90
Sincérité	8.30	7.18	1.13
Intelligence	8.41	7.36	1.04



Corrélation auto-perception / note réelle (attractivité) : 0.175

Réponse à Q4 :

Non, les gens ne peuvent pas prédire précisément leur valeur perçue. On observe une **surestimation systématique et significative** sur les trois attributs :

- Attractivité : surestimée de **+0.90 point** (7.08 perçu vs 6.18 réel)
- Sincérité : surestimée de **+1.13 point** (8.30 vs 7.18)
- Intelligence : surestimée de **+1.04 point** (8.41 vs 7.36)

La corrélation entre auto-perception et note réelle d'attractivité n'est que de **0.175** - très faible. Les gens ont donc une vision très imprécise de leur attractivité réelle. Ce phénomène s'appelle le **biais d'auto-complaisance** (*self-serving bias*) : on a naturellement tendance à se percevoir plus positivement que les autres ne nous voient.

12. Q5 - Mieux vaut-il être le premier ou le dernier rendez-vous de la soirée ?

"Pour obtenir un deuxième rendez-vous, est-il préférable d'être le premier speed date de la soirée ou le dernier ?"

```
In [13]: # Pour cette question, seules deux variables sont nécessaires : La position du
# rendez-vous dans la soirée ('order') et la décision du partenaire ('dec_o').
# Ces deux colonnes sont disponibles sur l'ensemble des lignes : on travaille donc
# sur toutes les observations exploitables, sans le filtrage propre à df_clean
# (qui supprimait des lignes pour des notes absentes, sans rapport avec la Q5).
q5 = df.dropna(subset=['order', 'dec_o']).copy()
print(f"Observations exploitables pour la Q5 : {len(q5)}")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# --- Graphique 1 : évolution du taux de 'Oui' position par position ---
order_rate = q5.groupby('order')['dec_o'].mean().reset_index()
sns.lineplot(x='order', y='dec_o', data=order_rate, color='steelblue', marker='o',
             markersize=5, ax=axes[0])
axes[0].set_title("Évolution du taux de 'Oui' au fil de la soirée")
axes[0].set_xlabel("Position du rendez-vous dans la soirée")
axes[0].set_ylabel("Taux moyen de décision positive")

# Ligne de tendance pour visualiser la direction générale
z = np.polyfit(order_rate['order'], order_rate['dec_o'], 1)
p = np.poly1d(z)
axes[0].plot(order_rate['order'], p(order_rate['order']),
             "r--", alpha=0.7, label='Tendance générale')
axes[0].legend()

# --- Graphique 2 : découpage de la soirée en trois tiers ---
max_order = q5['order'].max()
q5['phase'] = pd.cut(
    q5['order'],
    bins=[0, max_order * 0.33, max_order * 0.66, max_order],
    labels=['Début\n(1er tiers)', 'Milieu\n(2e tiers)', 'Fin\n(3e tiers)']
)
phase_rate = q5.groupby('phase', observed=True)['dec_o'].mean().reset_index()
sns.barplot(x='phase', y='dec_o', data=phase_rate,
            hue='phase', legend=False, palette=['green', 'orange', 'red'], ax=axes[1])
axes[1].set_title("Taux de 'Oui' selon la phase de la soirée")
axes[1].set_xlabel("Phase de la soirée")
axes[1].set_ylabel("Taux moyen de décision positive")
axes[1].set_ylim(0, 0.55)
for p_bar in axes[1].patches:
    axes[1].annotate(f'{p_bar.get_height():.1%}',
                     (p_bar.get_x() + p_bar.get_width() / 2., p_bar.get_height()),
                     ha='center', va='bottom', fontsize=12, fontweight='bold')

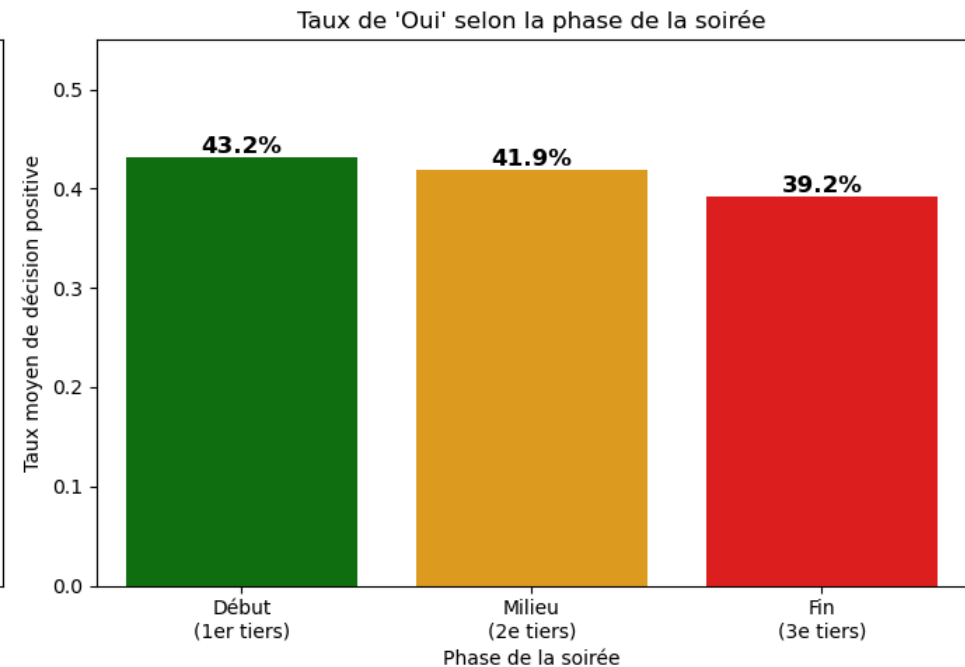
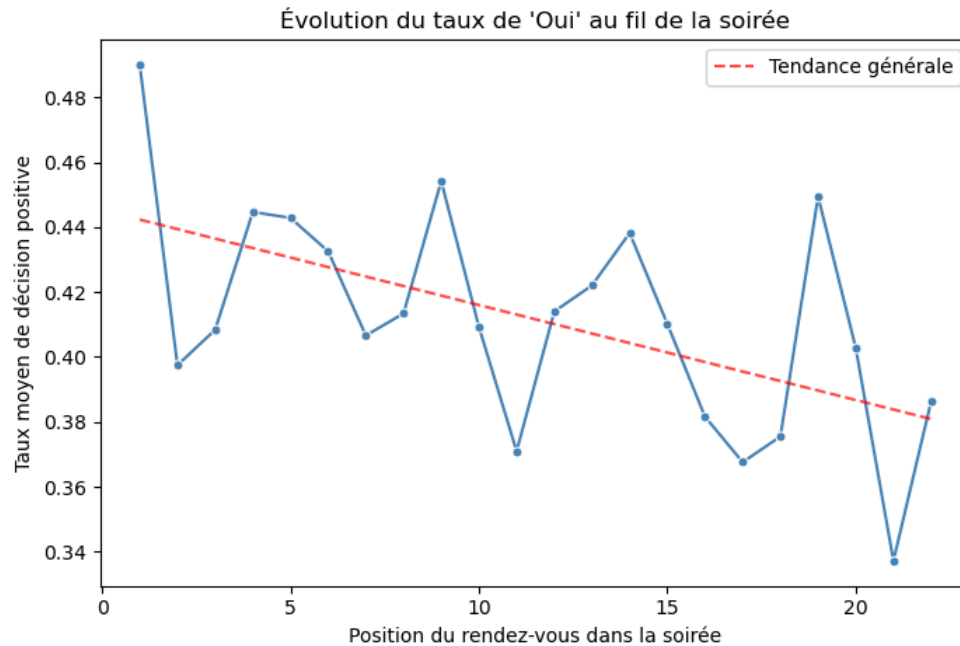
plt.tight_layout()
plt.show()

# --- Comparaison robuste : tout premier vs tout dernier rendez-vous de chaque soirée ---
# La position maximale varie d'une soirée (wave) à l'autre : on compare donc le
# premier rendez-vous (order = 1) au dernier rendez-vous propre à chaque soirée.
q5['max_in_wave'] = q5.groupby('wave')['order'].transform('max')
first_date = q5[q5['order'] == 1]['dec_o'].mean()
last_date = q5[q5['order'] == q5['max_in_wave']]['dec_o'].mean()

# Comparaison des tiers extrêmes (1er tiers vs 3e tiers)
debut = phase_rate.loc[phase_rate['phase'].str.startswith('Début'), 'dec_o'].values[0]
fin = phase_rate.loc[phase_rate['phase'].str.startswith('Fin'), 'dec_o'].values[0]
```

```
print(f"Taux de 'Oui' - tout premier rendez-vous de la soirée : {first_date:.1%}")
print(f"Taux de 'Oui' - tout dernier rendez-vous de la soirée : {last_date:.1%}")
print(f"Écart premier vs dernier : {(last_date - first_date) * 100:.1f} points")
print(f"Écart 1er tiers vs 3e tiers : {(fin - debut) * 100:.1f} points")
print(f"Corrélation position / décision : {q5['order'].corr(q5['dec_o']):.3f}")
```

Observations exploitables pour la Q5 : 8378



Taux de 'Oui' - tout premier rendez-vous de la soirée : 49.0%
Taux de 'Oui' - tout dernier rendez-vous de la soirée : 45.4%
Écart premier vs dernier : -3.6 points
Écart 1er tiers vs 3e tiers : -4.0 points
Corrélation position / décision : -0.031

Réponse à Q5 :

Il vaut mieux être rencontré en début de soirée, mais l'effet est modéré. On observe un déclin léger et régulier du taux d'acceptation au fil de la soirée :

- en découpant la soirée en trois tiers, le taux de « Oui » passe d'environ **43.2 % au début à 39.2 % à la fin**, soit un écart d'environ **-4 points** ;
- en comparant le tout premier rendez-vous au tout dernier de chaque soirée, on passe de **49.0 % à 45.4 %**, soit **-3.6 points**.

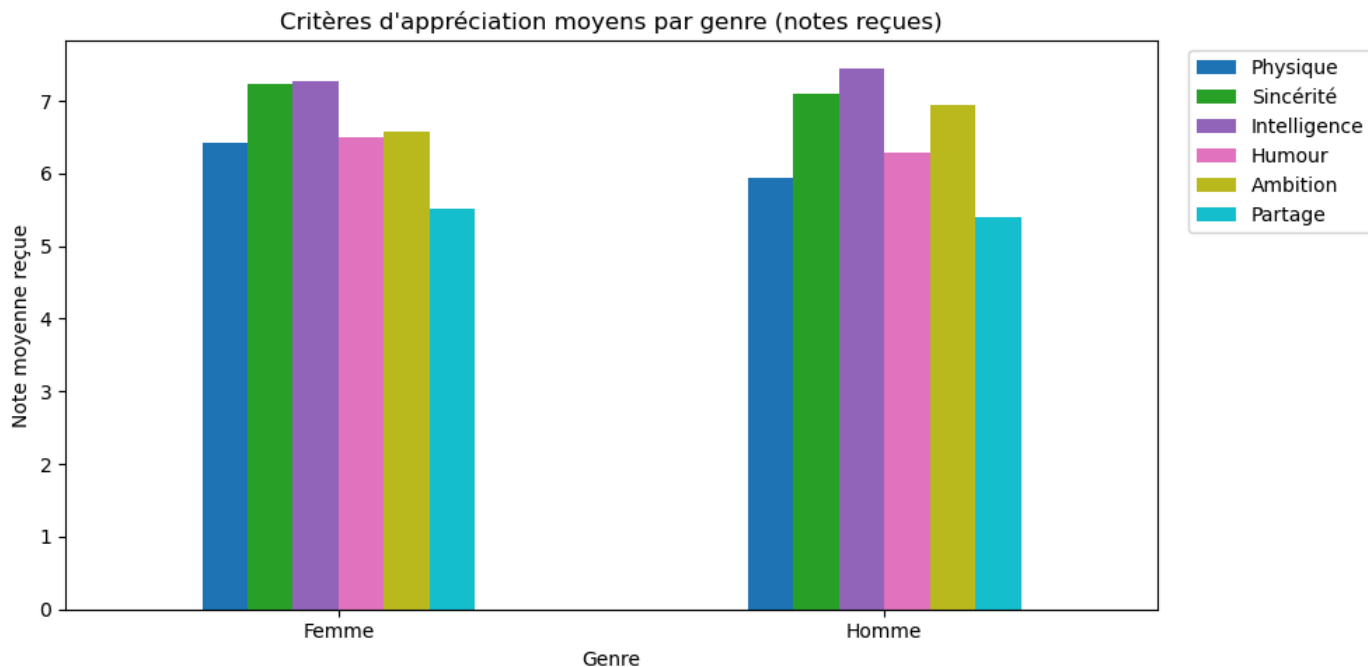
La direction est donc claire et cohérente entre les deux mesures, mais l'ampleur reste faible : la corrélation entre la position et la décision n'est que de **-0.03**, soit une perte d'environ 0.3 point de pourcentage par rendez-vous supplémentaire.

On ne parle donc pas d'un effet massif, mais d'une tendance descriptive régulière, compatible avec un phénomène de **fatigue décisionnelle** (*decision fatigue*) : au fil de rencontres répétées, la capacité d'évaluation se dégrade et les participants deviennent légèrement plus sélectifs. Ce biais cognitif est documenté en psychologie comportementale ; on le présente ici comme une hypothèse explicative et non comme une relation causale démontrée.

13. Analyse Approfondie : Profils par Genre et Corrélations

```
In [14]: # Comparaison des notes moyennes reçues par genre
gender_analysis = df_clean.groupby('gender')[score_cols].mean()

gender_analysis.plot(kind='bar', figsize=(10, 5), colormap='tab10')
plt.title("Critères d'appréciation moyens par genre (notes reçues)")
plt.ylabel("Note moyenne reçue")
plt.xlabel("Genre")
plt.xticks(rotation=0)
plt.legend(
    labels=['Physique', 'Sincérité', 'Intelligence', 'Humour', 'Ambition', 'Partage'],
    bbox_to_anchor=(1.02, 1), loc='upper left'
)
plt.tight_layout()
plt.show()
```



```
In [15]: # Graphique Radar : profil comparatif des notes reçues par genre
labels = np.array(['Physique', 'Sincérité', 'Intelligence', 'Humour', 'Ambition', 'Partage'])
stats_femmes = df_clean[df_clean['gender'] == 'Femme'][score_cols].mean().values
stats_hommes = df_clean[df_clean['gender'] == 'Homme'][score_cols].mean().values

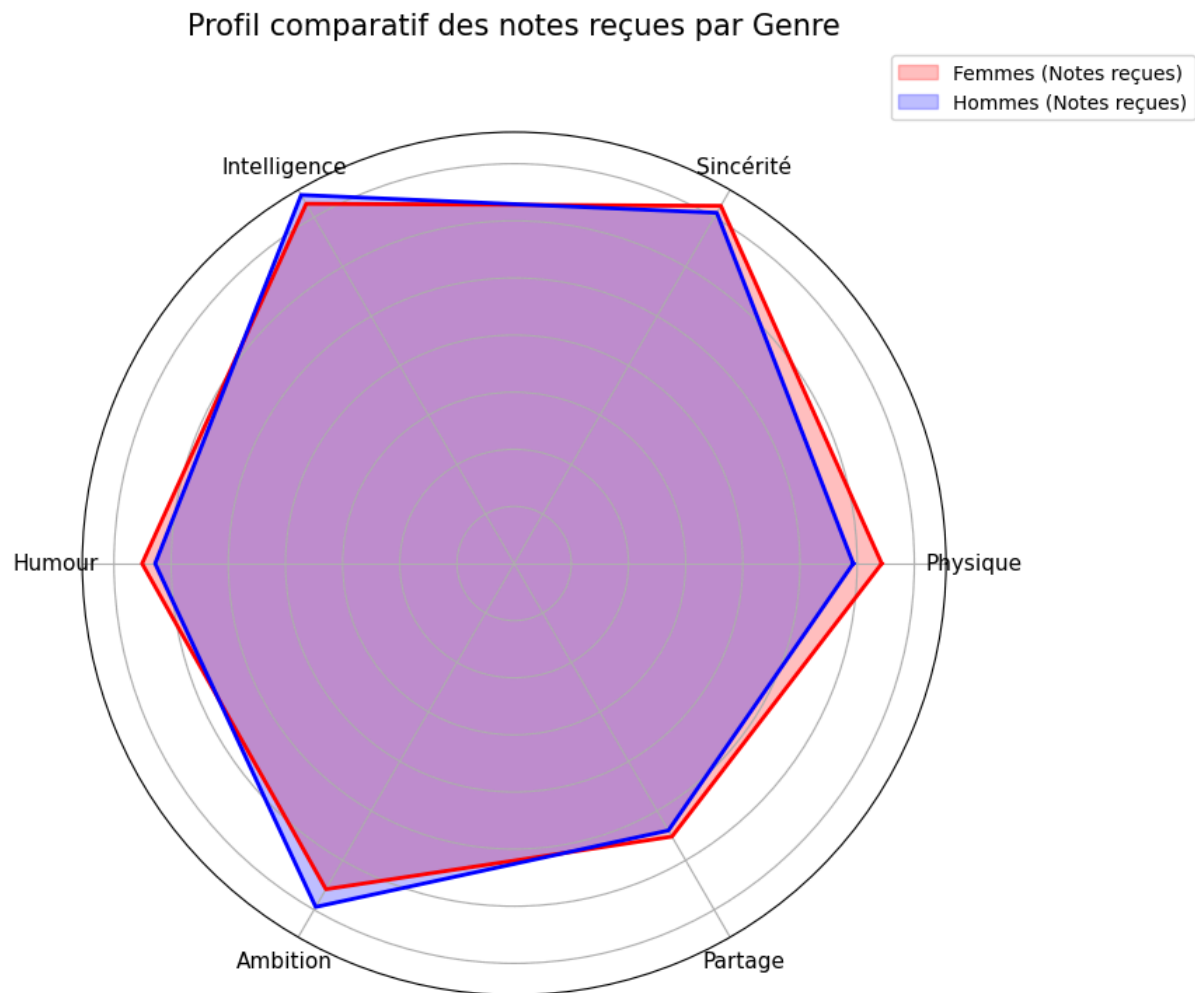
# Fermeture de la boucle du radar (répéter le premier point pour fermer le polygone)
stats_femmes = np.concatenate((stats_femmes, [stats_femmes[0]]))
stats_hommes = np.concatenate((stats_hommes, [stats_hommes[0]]))
angles = np.linspace(0, 2 * np.pi, len(labels), endpoint=False).tolist()
angles += angles[:1]

fig, ax = plt.subplots(figsize=(8, 8), subplot_kw=dict(polar=True))

ax.fill(angles, stats_femmes, color='red', alpha=0.25, label='Femmes (Notes reçues)')
ax.plot(angles, stats_femmes, color='red', linewidth=2)
ax.fill(angles, stats_hommes, color='blue', alpha=0.25, label='Hommes (Notes reçues)')
ax.plot(angles, stats_hommes, color='blue', linewidth=2)
```

```
ax.set_yticklabels([])
ax.set_xticks(angles[:-1])
ax.set_xticklabels(labels, fontsize=11)

plt.title("Profil comparatif des notes reçues par Genre", size=15, y=1.1)
plt.legend(loc='upper right', bbox_to_anchor=(1.3, 1.1))
plt.show()
```



Interprétation : La surface bleue (Hommes) est presque entièrement contenue dans la surface rouge (Femmes) : les femmes reçoivent de meilleures notes sur tous les critères. L'Intelligence et la Sincérité dominent pour les deux genres, tandis que l'Ambition reste systématiquement le point faible - cohérent avec nos résultats de Q1.

```
In [16]: # Heatmap de corrélation : chaque critère vs. La décision finale
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# --- Heatmap 1 : Corrélation critères → décision finale ---
correlation = (df_clean[score_cols + ['dec_o']]
               .corr()[['dec_o']]
               .sort_values(by='dec_o', ascending=False))
sns.heatmap(correlation, annot=True, cmap='coolwarm', fmt='.2f',
```

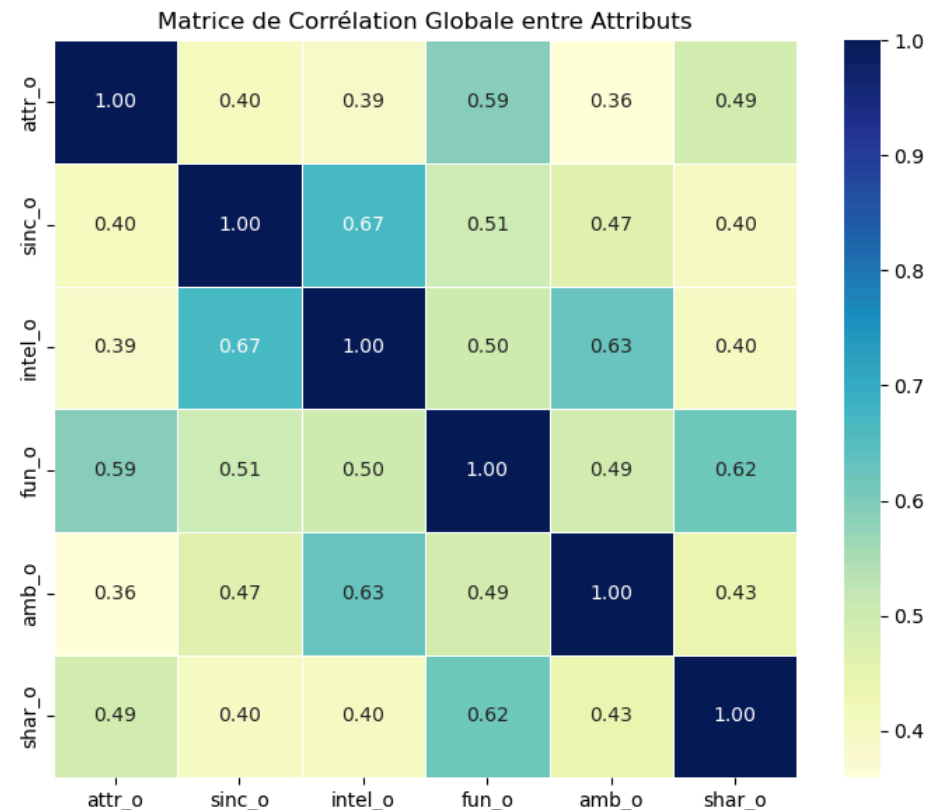
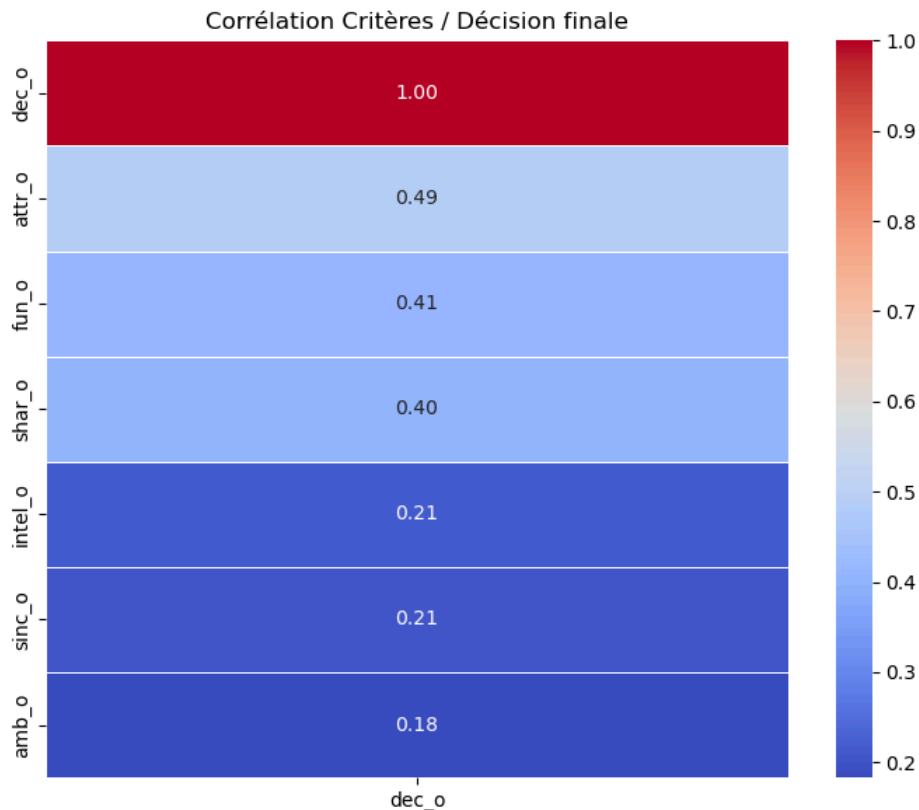
```

linewidths=0.5, ax=axes[0])
axes[0].set_title("Corrélation Critères / Décision finale")

# --- Heatmap 2 : Matrice de corrélation globale entre attributs ---
corr_globale = df_clean[score_cols].corr()
sns.heatmap(corr_globale, annot=True, cmap='YlGnBu', fmt='.2f',
            linewidths=0.5, ax=axes[1])
axes[1].set_title("Matrice de Corrélation Globale entre Attributs")

plt.tight_layout()
plt.show()

```



Interprétation :

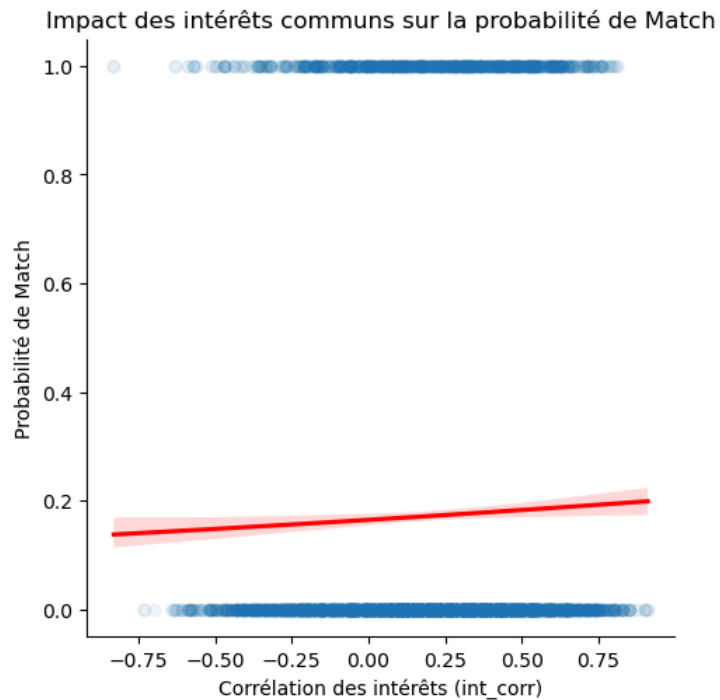
- **Heatmap gauche** : L'attractivité (attr_o , corr = 0.49) est le critère le plus corrélé à la décision finale, suivi de l'humour (0.41) et des intérêts partagés (0.40). La sincérité et l'ambition sont les moins influents (≈ 0.18-0.21).
- **Heatmap droite** : On observe un **effet de halo** entre l'attractivité et les autres traits - être jugé physiquement attirant booste inconsciemment les notes de personnalité. La forte corrélation entre fun_o et shar_o confirme que l'on s'amuse avec ceux qui nous ressemblent.

```

In [17]: # Régression Logistique : probabilité de match selon la corrélation des intérêts
# La régression logistique est adaptée car la variable cible (match) est binaire (0/1)
sns.lmplot(x='int_corr', y='match', data=df_clean,
           logistic=True, scatter_kws={'alpha': 0.05}, line_kws={'color': 'red'})
plt.title("Impact des intérêts communs sur la probabilité de Match")
plt.xlabel("Corrélation des intérêts (int_corr)")

```

```
plt.ylabel("Probabilité de Match")
plt.show()
```



Interprétation : La courbe logistique ascendante confirme que **plus les intérêts sont partagés, plus la probabilité de match augmente** - un facteur de réussite plus stable et plus durable que l'attractivité physique, et donc plus actionnable par un algorithme de matching.

14. Conclusion Générale et Recommandations Stratégiques

L'analyse complète de ce dataset permet de répondre aux 5 questions du sujet et de formuler des recommandations concrètes pour Tinder.

Synthèse des réponses aux 5 questions

Question	Réponse clé
Q1 - Attribut le moins désirable ?	Les priorités diffèrent selon le genre. Les hommes recherchent surtout l' attractivité (26.9/100), les femmes l' intelligence (21.0) et la sincérité (18.3). Le critère le moins valorisé est l' ambition chez les hommes (8.6) et le partage des intérêts chez les femmes (12.7).
Q2 - Attractivité déclarée vs. réelle ?	Les gens sous-déclarent l'importance du physique (22.5/100) mais c'est le critère le plus impactant en réalité (corr = 0.49). Biais de désirabilité sociale confirmé.
Q3 - Intérêts vs. même origine ?	Les intérêts communs ont un impact 3x plus fort (+3.3 pts de match) que l'origine raciale partagée (+1.0 pt).
Q4 - Auto-perception fiable ?	Non : surestimation systématique de +0.9 à +1.1 point sur tous les critères. Corrélation auto-perception/note réelle = 0.175 seulement.
Q5 - Premier ou dernier rendez-vous ?	Premier , avec un effet modéré : déclin régulier de 43.2 % (début) à 39.2 % (fin) en tertiles, et de 49.0 % à 45.4 % entre le tout premier et le tout dernier rendez-vous. Tendance compatible avec une fatigue décisionnelle.

Recommandations stratégiques pour Tinder

1. Valoriser le contenu visuel des profils

L'attractivité est le premier filtre de sélection ($\text{corr} = 0.49$ avec la décision). Tinder devrait continuer à mettre les photos au cœur de l'expérience et proposer des outils d'aide à la sélection de la meilleure photo.

2. Renforcer l'algorithme de matching par les passions

Les intérêts communs ont un impact 3x plus fort que l'origine raciale. L'ajout systématique de « Tags » de centres d'intérêt et leur pondération dans l'algorithme permettrait d'améliorer significativement le taux de match.

3. Limiter le volume de profils par session

La fatigue décisionnelle se traduit par un déclin léger mais régulier du taux d'acceptation au fil de la soirée (de l'ordre de -3.6 à -4 points entre le début et la fin). Introduire une limite quotidienne de swipes ou des pauses suggérées pourrait préserver la qualité des décisions et limiter ce phénomène d'évaluation dégradée.

4. Travailler la perception de soi des utilisateurs

Le biais d'auto-complaisance (+1 point de surestimation) crée des attentes irréalistes et génère des frustrations. Des fonctionnalités de feedback anonymisé ou de calibration du profil pourraient aider les utilisateurs à mieux se positionner sur le marché.